项目架构设计文档 - 第一阶段：开放世界底层构建

版本: 4.5 **状态:** 总体概览确认阶段 **目标:** 确立超大开放世界底层系统的功能需求与核心技术路线，为后续具体结构设计提供定性依据。

前言: 本阶段旨在构建一个能够支撑“2d开放世界探索 + 策略/RTS元素 + 沙盒建造”三种玩法无缝融合的底层世界框架。核心在于解决超大规模地图的性能问题，以及在 2D 俯视角下实现具有立体感和策略深度的高度地形系统。

1. 实现要求 (Implementation Requirements)

本节定义了第一阶段必须交付的游戏功能特性。

A. 地图性质与规模 (Map Nature & Scale)

- **超大有限世界:** 地图并非无限生成，而是一个具有固定边界的超大矩形世界。
- **程序化随机生成 (Procedural Generation):**
  - **基于种子:** 世界并非预先设计好的静态地图，而是基于 **随机种子 (Seed)** 程序化生成的。
  - **重玩价值:** 每一次新游戏都可以生成完全不同的地形地貌和资源分布。
  - **边界控制:** 虽然内容随机，但生成算法会严格遵循配置的边界范围，超出范围的区域生成为无效或海洋边界。
- **配置核心 (Configurable Core):**
  - **配置单位:** 地图的最终大小完全由 **区块数量 (Chunk Count)** 决定。例如 `WorldSize = Vector2i(6000, 6000)` 。
  - **自适应性:** 底层系统根据配置的区块数量，自动推导出总像素尺寸和 Region 索引范围。**同时支持长宽不一致的非正方形地图配置（即长与宽可以不同），系统需能适应任意矩形比例。**
- **基础度量衡 (Metrics):**
  - **瓦片 (Tile):** `16 x 16` 像素。基础的视觉和交互单位。
  - **区块 (Chunk):** `32 x 32` 瓦片（即 `512 x 512` 像素）。内存加载、渲染批次处理及网络同步的最小单位。
  - **区域 (Region):** `32 x 32` 区块（即 `16,384 x 16,384` 像素）。磁盘索引单位。
- **开发测试规模 (Dev/Test Dimensions):** 为了便于开发测试并保证国战级的战略纵深，采用以下整洁的整数规模：
  - **配置规模: 6,000 x 6,000 区块** (总计 3600 万个区块)。
  - **瓦片总数:** 边长 **192,000 瓦片** (19.2万 x 19.2万)。
  - **像素总宽:** 边长 **3,072,000 像素** (约 307万 px)。
  - **区域文件:** 约 **188 x 188** 个 Region 文件 (总计约 3.5万个文件)。

    - **规模定义:** 这是一个极具宏大感的规模，横穿地图需耗费大量游戏时间，足以容纳多个国家势力的疆域分布。
- **无缝漫游:** 玩家从世界一端移动到另一端，通过区块流式加载 (Streaming) 技术实现无缝衔接，无任何加载读条界面。

B. 伪高度立体地形 (Pseudo-Height Terrain)

为了在 2D 视角下提供“翻山越岭”的沉浸感，地图必须具备逻辑和视觉上的高度属性：

- **离散高度层级:** 地形拥有绝对高度属性 (Elevation)，如 H0, H1, H2。
- **视觉表现:**
    - **悬崖与坡度:** 高度差产生的边缘自动渲染为悬崖或斜坡。
    - **大气透视与阴影:** 高层地貌对低层投射阴影，且高层纹理通过色调微调体现深度。
- **移动反馈:** 地形高度直接影响玩法。上坡减速（模拟攀登），高度差过大形成物理阻挡（绝壁）。

C. 环境交互系统 (Environmental Interaction)

世界充满了可互动的自然元素，玩家可以改变环境本身。

- **自然物体交互:** 地图上分布着草丛、树木、岩石矿脉、河流等独立对象。
    - **采集/破坏:** 玩家可以砍伐树木获取木材，开采岩石获取石料，清除杂草等。
    - **状态变化:** 树木被砍倒后会消失；草地被清除后会露出下层泥土。
- **地形改造 (Terraforming):** 地形本身也是可交互对象。
    - **开凿与堆砌:** 玩家可以对高山地块进行“挖掘”，使其高度层级 (Elevation) 降低（例如将 H2 的山坡挖成 H1 的平地）；反之也可以填土堆高。
    - **河流与水体:** 能够改变地形引导水流或填埋水坑。

D. 建筑营造系统 (Building Construction)

独立于自然环境的建造体系，支持微观的精细构建。

- **微观构建 (Micro-Building):** 采用类似《Minecraft》的单方块构建模式。玩家可以逐个放置墙壁、地板、门窗、家具。
- **独立的楼层高度系统 (Storey System):**
    - **区别于地形:** 地形采用自然起伏的 `Elevation` 系统，而建筑采用标准的 `Storey` （楼层）系统。
    - **固定层高:** 每一层楼（1F, 2F）具有固定的逻辑高度和视觉高度偏移，不受地形微小起伏影响，但必须建立在稳固的地形或地基之上。
- **放置与环境检测:**
    - **地形依赖:** 建筑方块的放置必须检测目标地块的地形属性。例如，地基必须放置在平整的表面；某些建筑无法放置在陡峭的悬崖边缘或水面上。

    - **障碍检测:** 放置前必须确认目标空间内没有树木、岩石等自然障碍物（玩家需先清理环境）。
- **二元统一预留:**
    - **物理层:** 建筑由独立方块组成，可物理破坏。
    - **逻辑层:** 系统需能够识别一组方块为一个“功能建筑”（如兵营），为后续策略玩法预留接口。

E. 动态世界基础

- **持久化:** 玩家对地形高度的修改、自然物体的破坏、人工建筑的营造都必须被永久保存。
- **统一承载:** 无论是自然生成的山体还是玩家建造的城堡，在底层数据中统一管理，确保无缝衔接。

F. 存档管理系统 (Save System Management)

- **基本功能:**
    - **存档/读档:** 完整支持游戏状态的序列化与反序列化，包括世界种子、玩家位置、所有已修改的区块数据。
    - **多存档槽:** 支持创建多个独立的世界存档。
- **路径配置 (Custom Save Path):**
    - **非写死路径:** 存档文件的存储位置**不强制绑定**在 Godot 默认的 `user://` 目录下。
    - **自定义支持:** 系统需支持通过配置文件（`config.ini`）或启动参数指定外部绝对路径（例如 `D:/MyGameSaves/`）。方便玩家进行存档备份、云同步或位置迁移。

2. 技术总览 (Technical Overview)

本节确立为了实现上述功能，我们将采用的核心技术路线和架构模式。

A. 核心架构模式：数据与表现分离 (Data-View Separation)

- **理念:** 严格区分“世界数据”与“场景节点”。
- **策略:**
    - **数据 (Model):** 整个世界的真理（地形高度、物体ID、建筑数据）存储在内存数据结构或磁盘文件中。
    - **表现 (View):** 仅在玩家视野范围内实例化 Godot 节点（TileMapLayer, Sprite2D），作为数据的视觉投影。

B. 地图生成技术 (Map Generation Technology)

本部分描述如何通过数学算法构建世界的初始状态，是后续地图管理和存储的数据源头。

- **技术选型:** 基于 Godot 内置的 `FastNoiseLite` 噪声库，配合自定义的确定性生成管线。
- **生成策略:**

    - **确定性生成 (Deterministic Generation):** 只要 **随机种子 (Seed)** 相同，无论何时何地计算，特定坐标 `(x, y)` 的地形结果永远一致。这允许我们在不存储 300万像素地图初始数据的情况下，实现按需生成。
    - **基础地形 (Base Terrain):** * **高度图 (Height Map):** 使用大尺度噪声 (Perlin/Simplex) 决定基础地形的 `Elevation` (平原、山脉、深坑)。
        - **无生物群落 (No Biomes):** 第一阶段不实现复杂的温度/湿度生物群落系统，专注于基础地形高度的生成与平滑过渡。
    - **细节分布 (Scatter):** 使用高频噪声或泊松盘采样 (Poisson Disk Sampling) 决定树木、岩石的分布密度和位置。
- **生成时机 (Generation Timing):**
    - **即时生成 (On-the-Fly):** 当 **流式加载器 (Level 3)** 尝试读取一个区块，且磁盘中不存在对应的 Region 存档数据时，系统立即调用生成算法实时计算该区块数据。
    - **存档优先:** 若磁盘中存在玩家之前修改过并保存的 Region 数据，则直接读取存档，**跳过**生成步骤。

C. 地图管理：四级对称流水线 (Symmetrical Four-Stage Pipeline)

为了最大化性能并明确资源管理边界，我们采用“同心圆”式的分级加载策略。以玩家当前所在的区块坐标为中心，向外辐射定义四个区域。

核心机制：迟滞区间 (Hysteresis Buffer)

- **原理:** 每个层级都有一个较小的 `加载半径 (R_load)` 和一个较大的 `卸载半径 (R_unload)`。
- **时机:**
    - **加载 (Upgrade):** 只有当区块进入 `R_load` 时，才执行资源分配（读盘/实例化/显示）。
    - **卸载 (Downgrade):** 只有当区块移出 `R_unload` 时，才执行资源释放（隐藏/销毁/删除）。
    - **缓冲区:** 位于 `R_load < 距离 <= R_unload` 之间的区块保持当前状态不变。这防止了玩家在边界往复移动时引发频繁的系统抖动。

Level 1: 活跃层 (Active Layer) - 屏幕可见区

- **范围定义:**
    - **加载触发:** 距离 $\leq$ **1 个区块** (3x3 九宫格)。
    - **卸载触发:** 距离 $>$ **2 个区块**。
- **资源状态:** 内存中有数据，场景中有可见且活跃的 Godot 节点。
- **行为:** 负责渲染和实时交互逻辑。

Level 2: 就绪层 (Ready Layer) - 显存缓冲

- **范围定义:**
    - **加载触发:** 距离 $\leq$ **2 个区块** (5x5 区域)。
    - **卸载触发:** 距离 $>$ **3 个区块**。

- **资源状态:** 内存中有数据，场景中有隐藏且冻结的 Godot 节点。
- **行为:** 预先实例化节点以消除卡顿，离开时销毁节点释放显存。

Level 3: 数据层 (Data Layer) - 内存缓冲

- **范围定义:**
  - **加载触发:** 距离 $\leq$ **8 个区块** (17x17 区域)。
  - **卸载触发:** 距离 $>$ **10 个区块**。
- **资源状态:** 仅内存中驻留纯数据对象，无任何 Godot 节点。
- **行为:** 负责从硬盘预读取数据，离开时负责将修改过的数据回写硬盘。

Level 4: 磁盘区 (Disk Zone)

- **范围定义:** 距离 $>$ **10 个区块** 的所有区域。
- **状态:** 数据仅存在于硬盘文件中，内存和显存中均无任何残留。

D. 存储设计：分区域聚合存储 (Aggregated Region Storage)

本部分描述数据持久化的核心策略，确保海量数据的高效读写与安全性。

- **核心策略: 分区域聚合 (Data Aggregation)**
  - **概念:** 为了解决操作系统无法高效处理数百万个小文件的问题，系统将地图划分为更大的逻辑单位——**区域 (Region)**。
  - **机制:** 每个区域作为一个独立的存储容器，内部包含并索引了 `32 x 32` **(1024个)** 连续的**区块 (Chunk)** 数据。磁盘 I/O 操作始终以“区域”为单位进行文件句柄管理，而在内部以“区块”为单位进行数据定位。
- **路径配置 (Path Configuration)**
  - **根目录:** 存储根目录由 `SaveSystem.save_path` 变量决定。
  - **初始化:** 游戏启动时，优先读取配置文件中的路径；若未配置，则回退到 `user://saves/` 。
  - **结构:** `{save_path}/{world_name}/regions/r.x.y.data` 。
- **脏数据管理 (Dirty Data Management)**
  - **概念:** 内存中的区块数据对象包含一个状态标记。
  - **机制:** 当玩家对地图进行任何修改（如破坏方块、改变地形）时，对应区块被标记为 **“脏 (Dirty)”**。只有被标记为“脏”的数据才需要执行写入操作；未修改的数据在卸载时直接从内存丢弃，无需 I/O。
- **持久化触发机制 (Persistence Triggers)** 系统定义了三种触发数据写入的时机：
  1. **卸载触发 (Unload Sync):** 当一个“脏”区块因为玩家远离而移出 **数据层 (Level 3)** 时，它不会立即销毁，而是被移交到后台写入队列。
  2. **定时触发 (Auto-Save):** 系统维护一个定时器（例如每几分钟），周期性地扫描内存中所有“脏”区块，将它们复制到写入队列，并重置内存中的标记。这防止了程序崩溃导致长时间进度的丢失。

3. **强制触发 (Force Save):** 在玩家手动存档或退出游戏时，强制阻塞主线程，直到所有数据完成写入。

- **异步写入流水线 (Asynchronous I/O Pipeline)** 为了杜绝保存时的游戏卡顿，所有写入操作遵循以下流程：
  - **快照复制:** 主线程仅负责将需要保存的数据进行快速深拷贝（Snapshot），随后立即继续游戏逻辑。
  - **后台序列化:** 后台线程负责将快照数据转换为二进制或文本格式。
  - **原子性写入:** 数据写入磁盘时采用“写入临时文件 -> 校验完整性 -> 重命名覆盖”的原子操作模式，确保即使在写入过程中断电，存档文件也不会损坏。

E. 渲染技术：双重高度混合渲染

- **技术选型:** 基于 Godot 的 `TileMapLayer` 和 `Y-Sort` 机制。
- **混合高度实现:**
  - **地形渲染 (Terrain Rendering):** 采用 **悬崖图块 + 视觉着色 (Autotile + Shader)** 复合方案。
    - **核心机制:** 地形高度 (`Elevation`) 的视觉表现由 **结构** (悬崖) 和 **氛围** (着色) 共同构成。
    - **悬崖自动图块 (Cliff Autotile - Phase 1 重点):**
      - **功能:** 处理高度差产生的物理边缘。通过检测相邻地块的 `Elevation` 差值，自动绘制悬崖面、斜坡或阴影边缘图块。
      - **第一阶段状态: 核心实现**。第一阶段将集中精力完成自动拼接算法，确保地形在几何逻辑上闭合且视觉正确。
    - **高度感知 Shader (Atmospheric Shader - Future Proofing):**
      - **功能:** 通过检测地块的 `Elevation` 值，动态调整图块的亮度或色调。
      - **第一阶段状态: 数据预留**。第一阶段不编写具体 Shader 代码，但需在 TileData 中预留 Custom Data 接口，确保未来接入 Shader 时无需重构数据层。
  - **建筑渲染 (Building Rendering):** 采用 **垂直偏移 (Vertical Offset)** 方案。
    - 在地形基础之上，建筑楼层 (`Storey`) 渲染时叠加固定的 Y 轴偏移（例如每层楼向上偏移 16px）。
    - 这使得人工建筑在视觉上具有明显的立体感和垂直遮挡关系。